

HTML Learning Portal

Unit 3 Study Guide

Cascading Style Sheets

18 topics · condensed key concepts · exam-style practice questions with answers

How to use this guide

Part 1 is a condensed, topic-by-topic reference — read it as a quick refresher, not a first-time introduction (for that, use the full tutorials on the site). Part 2 is a set of exam-style practice questions covering the whole unit, in several formats you're likely to see on a real exam: short answer, differentiate-between, write-the-code, fill-in-the-blank, and true/false. Answers are collected at the end so you can self-check without seeing them while you attempt the questions.

by Dr. Puneet Thapar — Associate Professor, School of Computing and Artificial Intelligence, Lovely Professional University

Part 1 — Quick Reference

26. Introduction to CSS

- CSS separates presentation from content: HTML says what something is, CSS says how it looks.
- Three ways to add CSS: inline (style attribute), internal (<style> in <head>), external (<link rel="stylesheet">) — external is best practice.
- A rule is selector { property: value; }. On a specificity tie, the later rule in the stylesheet wins.

27. CSS Selectors and Specificity

- Basic selectors: type (p), class (.name), ID (#name), universal (*).
- Combinators: descendant (space), child (>), adjacent sibling (+), general sibling (~).
- Specificity order (low→high): element/pseudo-element < class/attribute/pseudo-class < ID < inline style; !important overrides all of this.

28. Colors and Backgrounds

- Color formats: keyword names, hex, rgb()/rgba(), hsl()/hsla().
- background-image + background-size/position/repeat control an image fill; linear-gradient()/radial-gradient() generate gradients with no image file.
- opacity fades the whole element (including children); an rgba()/hsla() alpha channel only fades that one color value.

29. Units and Measurements

- px is absolute (fixed); em is relative to the parent's font-size (compounds when nested); rem is relative to the root font-size (predictable).
- vw/vh/vmin/vmax are relative to the viewport; percentages are relative to the parent's corresponding property.
- calc() mixes units in one expression, e.g. calc(100% - 40px).

30. Typography

- font-family takes a fallback list ending in a generic family (serif/sans-serif/monospace).
- font-weight can be named or numeric (100–900); a unitless line-height (e.g. 1.6) scales safely with font-size.
- @font-face loads a custom font file so text isn't limited to system fonts.

31. Text Styling

- text-align (left/right/center/justify); text-decoration (underline/line-through + sub-properties for color/style/thickness).
- text-transform changes displayed case only, without altering the actual HTML text content.
- The 3-property ellipsis pattern: white-space:nowrap + overflow:hidden + text-overflow:ellipsis, all required together.

32. Box Model

- Every box = content + padding + border + margin, from the inside out.
- content-box (default): width/height set only the content area, padding/border add on top. border-box: width/height include padding/border.
- Adjacent vertical margins collapse to the larger value, not the sum. margin/padding shorthand accepts 1, 2, 3, or 4 values.

33. Positioning Techniques

- static = default flow; relative = flow + offset + becomes anchor for absolute children.

- absolute = removed from flow, positions relative to nearest positioned ancestor; fixed = relative to the viewport, stays on scroll; sticky = relative until a scroll threshold, then behaves like fixed within its container.
- z-index resolves overlap order among positioned elements — only has an effect on non-static elements.

34. Flexbox Layout

- display:flex turns direct children into flex items along a main axis (flex-direction: row/column).
- justify-content aligns along the main axis; align-items aligns along the cross axis; align-self overrides one item.
- flex-wrap lets items flow onto multiple lines; flex-grow/flex-shrink/flex-basis (the flex shorthand) control resizing; gap adds spacing.

35. CSS Grid Layout

- display:grid + grid-template-columns/rows define column/row tracks; fr divides remaining space proportionally.
- grid-column/grid-row place and span items; grid-template-areas sketches a named layout.
- repeat(auto-fit, minmax(120px, 1fr)) builds a grid that reflows its own column count with no media query.

36. Pseudo Classes

- :hover, :focus, :active respond to user interaction in real time, no JS needed.
- :first-child, :last-child, :nth-child() select by sibling position (e.g. :nth-child(odd) for zebra-stripping).
- :checked, :disabled, :required, :valid/:invalid reflect a form control's current state; :not() excludes a case from a rule.

37. Pseudo Elements

- A pseudo-element (::) targets a sub-part of an element with no HTML node of its own — distinct from a pseudo-class (:).
- ::before/::after insert generated content via the required content property, even content: "" for a purely visual element.
- ::first-letter/::first-line style just the first character/line; ::placeholder and ::selection style input placeholders and selected text.

38. Attribute Selectors

- [attr] matches presence; [attr="value"] matches an exact value.
- [attr^="value"] starts-with, [attr\$="value"] ends-with, [attr*="value"] contains, [attr~="value"] whole word in a space-separated list.
- Adding a space + i before the closing bracket makes the match case-insensitive.

39. Form Styling

- border/padding/border-radius reshape inputs; a :focus style (border-color/box-shadow) gives clear active-field feedback.
- appearance:none strips native OS styling as a base for fully custom buttons/selects.
- A native select's dropdown arrow and checkbox/radio glyphs are notoriously hard to restyle consistently across browsers.

40. CSS Variables

- A custom property is declared --name: value; (usually on :root) and read with var(--name).
- A variable is available to the element it's declared on and all descendants; redeclaring it on a nested element overrides it locally.
- var(--name, fallback) supplies a default; unlike a Sass/Less variable, a CSS custom property is live and can change at runtime.

41. Responsive Web Design

- Mobile-first uses min-width media queries to add complexity as the screen grows; pick one convention (mobile-first or desktop-first) consistently.
- `object-fit:cover` fills a box without distorting aspect ratio; `aspect-ratio` reserves space before media loads.
- `prefers-color-scheme` (dark mode), `orientation`, and `@media print` are other useful media features; `@container` queries respond to a component's own container size.

42. CSS Animations

- `@keyframes` defines named style states at percentage checkpoints (0%–100%, or from/to).
- `animation-duration`, `-timing-function`, `-iteration-count`, `-direction`, `-fill-mode` (and the animation shorthand) control playback.
- Prefer animating `transform/opacity` (smooth, no layout recalculation) over `width/top`; respect `prefers-reduced-motion`.

43. CSS Transitions

- A transition needs a trigger (like `:hover` or a class toggle) and moves from state A to state B — unlike an animation, it doesn't run automatically or loop.
- The transition shorthand order is: property, duration, timing-function, delay.
- Comma-separate multiple property transitions rather than defaulting to `transition:all`, which can be less predictable and costlier to render.

Part 2 — Exam Practice Questions

Section A — Short Answer

- Q1. List the three ways CSS can be added to a page, and state which is generally recommended.
- Q2. Explain why rem is often preferred over em for consistent font sizing.
- Q3. What is the difference between opacity and an alpha value inside rgba()?
- Q4. Explain the difference between content-box and border-box in box-sizing.
- Q5. What condition allows position: sticky to behave like position: fixed?
- Q6. What does the fr unit represent in CSS Grid?
- Q7. What is required for a ::before or ::after pseudo-element to render anything at all?
- Q8. Why are CSS custom properties (variables) more powerful at runtime than Sass/Less variables?
- Q9. Why should transform and opacity be preferred over width/top for animation?
- Q10. What is the key difference between a CSS transition and a CSS animation?

Section B — Differentiate Between

- Q11. [Differentiate] Descendant combinator (space) vs. child combinator (>)
- Q12. [Differentiate] justify-content vs. align-items in Flexbox
- Q13. [Differentiate] Pseudo-class (e.g. :hover) vs. pseudo-element (e.g. ::before)
- Q14. [Differentiate] [attr^="value"] vs. [attr*="value"]
- Q15. [Differentiate] Mobile-first vs. desktop-first media query strategy

Section C — Write the Code

- Q16. [Code] Write a CSS rule that turns every direct child of a red, using the child combinator.
- Q17. [Code] Write a linear-gradient background going from #c4406b to #211f2e at a 135deg angle.
- Q18. [Code] Write a Flexbox rule that centers items both horizontally and vertically inside a container.
- Q19. [Code] Write a CSS Grid rule for a responsive card layout using repeat(auto-fit, minmax(200px, 1fr)).
- Q20. [Code] Declare a CSS custom property --brand-color on :root and use it to set a button's background.
- Q21. [Code] Write a @keyframes block named fadeIn that animates opacity from 0 to 1, and apply it to a class with a 1-second ease animation.

Section D — True or False

- Q22. [T/F] An ID selector has higher specificity than a class selector.
- Q23. [T/F] Adjacent vertical margins between block elements always add together.

Q24. [T/F] z-index has an effect on an element with position: static.

Q25. [T/F] A CSS transition can run automatically without any trigger, just like an animation.

Q26. [T/F] text-transform: uppercase permanently changes the underlying HTML text content.

Answer Key

- A1.** Inline (style attribute), internal (<style> block in <head>), and external (a linked .css file via <link>). External is generally recommended — it's cacheable, reusable across pages, and keeps markup clean.
- A2.** em compounds against each nested parent's font-size and can grow unpredictably; rem always refers back to the root <html> font-size regardless of nesting depth, staying consistent.
- A3.** opacity fades the entire element, including its text and all children, as a whole; an alpha channel in rgba()/hsla() only makes that one specific color value (e.g. just the background) partially transparent.
- A4.** content-box (default): width/height set only the content area, with padding and border added on top, making the rendered box larger than the set width. border-box: width/height include padding and border, so the set size is the final rendered size.
- A5.** position: sticky behaves like relative until the element would scroll past a given offset (like top: 0) within its containing block, at which point it sticks in place like fixed — but only within that container's bounds.
- A6.** fr represents a fraction of the grid container's remaining free space, letting tracks share leftover space proportionally (2fr is always twice as wide as 1fr).
- A7.** A content property (even content: "", empty) is required — without it, the pseudo-element won't render at all.
- A8.** A Sass/Less variable is substituted with its value at build time and is then fixed; a CSS custom property is live in the browser and can be changed at runtime — e.g. redefined inside a media query or updated with JavaScript — letting one change cascade everywhere it's used.
- A9.** transform and opacity can be animated by the browser without recalculating page layout; properties like width or top force the browser to repeatedly recompute layout on every frame, which can look janky.
- A10.** A transition requires a state change (like :hover or a class toggle) to move from one value to another one time; an animation (via @keyframes) can run automatically on page load and loop through many defined steps without any trigger.
- B1.** The descendant combinator (space) matches an element nested anywhere inside another, at any depth; the child combinator (>) matches only direct children, one level down.
- B2.** justify-content aligns flex items along the main axis (the row/column direction the container is flowing); align-items aligns them along the cross axis, perpendicular to that.
- B3.** A pseudo-class (single colon, e.g. :hover) matches an element based on a state or position it's already in; a pseudo-element (double colon, e.g. ::before) targets and can insert content into a sub-part of an element that has no node of its own in the HTML.
- B4.** [attr^="value"] matches attribute values that start with the given string; [attr*="value"] matches values containing that string anywhere within them.
- B5.** Mobile-first writes base styles for small screens first and adds min-width media queries to layer on complexity as the screen grows; desktop-first starts with the full desktop layout and uses max-width queries to simplify it down for smaller screens.
- C1.** `ul > li { color: red; }`
- C2.** `background: linear-gradient(135deg, #c4406b, #211f2e);`
- C3.** `.center { display: flex; justify-content: center; align-items: center; }`
- C4.** `.grid { display: grid; grid-template-columns: repeat(auto-fit, minmax(200px, 1fr)); gap: 16px; }`
- C5.** `:root { --brand-color: #c4406b; } .btn { background: var(--brand-color); }`
- C6.** `@keyframes fadeIn { from { opacity: 0; } to { opacity: 1; } } .box { animation: fadeIn 1s ease; }`
- D1.** True — ID selectors outrank class selectors in the specificity hierarchy.
- D2.** False — adjacent vertical margins collapse to the larger of the two, not the sum.
- D3.** False — z-index only has an effect on elements with a position value other than static.

D4. False — a transition requires a state-change trigger; only a `@keyframes` animation can run automatically without one.

D5. False — `text-transform` is purely visual; the underlying HTML text content is unchanged.